ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/387025687

Feasibility of Machine Learning-Enhanced Detection for QR Code Images in Email-based Threats

Conference Paper · October 2024

DOI: 10.1109/CARS61786.2024.10778732

CITATION
1

READS
79

2 authors:

Jason Ford
University of South Carolina Aiken
4 PUBLICATIONS **3** CITATIONS

SEE PROFILE

Hala Strohmier Berry
University of South Carolina Aiken
12 PUBLICATIONS **41** CITATIONS

SEE PROFILE

All content following this page was uploaded by Jason Ford on 13 December 2024.

The user has requested enhancement of the downloaded file.

Feasibility of Machine Learning-Enhanced Detection for QR Code Images in Email-based Threats

Jason Ford
Computer Science, Engineering and Mathematics
University of South Carolina Aiken
Aiken SC USA
jasonsf@usca.edu

Hala Strohmier Berry
Computer Science, Engineering and Mathematics
University of South Carolina Aiken
Aiken SC USA
hala.strohmier@usca.edu

***Abstract*—As QR codes become increasingly common in digital communication, cybercriminals have seized upon this technology as a vehicle for sophisticated URL-based email phishing attacks. These malicious QR codes, embedded within email messages, are designed to deceive recipients into revealing sensitive information. The primary challenge for cybersecurity vendors is efficiently detecting and analyzing these QR codes at scale, a task that is both computationally demanding and challenging within the high-throughput environment of email servers.**

This research investigates the application of convolutional neural networks (CNNs) to automate the detection of QR codes embedded in email images, addressing a growing vector for phishing attacks by integrating advanced image recognition techniques into existing email security frameworks. Through iterative development and refinement, a CNN model was designed to accurately differentiate between benign and malicious QR codes. The experimentation process revealed that while the model achieved high accuracy in early stages, it also encountered issues with overfitting as complexity increased, underscoring the need for careful balance in training processes.

The study concludes with a proof of concept that demonstrates the effectiveness of CNNs in enhancing email security systems. It also emphasizes the importance of continuous model adaptation to address the evolving nature of phishing threats. This work represents a significant step toward scalable and efficient solutions for detecting QR code-based phishing attacks in the dynamic cybersecurity landscape.

***Keywords*—QR code, phishing, convolutional neural networks, email security, threat detection, cybersecurity**

I. Introduction

Quick Response (QR) codes have become a ubiquitous part of modern digital interactions, offering a convenient method for encoding and sharing information. Originally developed for tracking vehicle parts during manufacturing, QR codes have evolved into a versatile tool used across various sectors, including marketing, healthcare, logistics, and personal transactions. Their ability to store a vast amount of data in a compact, easily scannable format has made them particularly popular. The widespread use of QR codes is further bolstered by the prevalence of smartphones equipped with built-in QR code readers, making the scanning process seamless and immediate.

The rise in QR code usage has coincided with growing concerns about their security vulnerabilities. Cybercriminals increasingly exploit QR codes to deliver malicious payloads, initiate phishing attacks, or redirect users to fraudulent websites. These threats highlight a critical need for robust security measures and heightened user awareness to prevent unauthorized access and data breaches. As a component of the Internet of Things (IoT) perception layer, QR codes facilitate the collection and processing of data, making them integral to various IoT applications. Niu et al. [1] highlight that this integration also positions QR codes as potential entry points for cyberattacks, compromising the integrity of the systems they support.

Traditional email security solutions primarily focus on analyzing text and URLs embedded in emails, relying heavily on keyword matching and heuristic analysis to detect threats. However, these methods are often ineffective at identifying malicious content embedded within images, such as QR codes, which can easily evade detection. This vulnerability in many contemporary email security systems is increasingly exploited by cybercriminals, presenting a growing challenge for cybersecurity defenses. Existing methods fall short because they are not equipped to analyze image-based content, leaving a significant gap in the detection of these threats.

In response to this new threat vector, cybersecurity researchers have begun exploring advanced techniques to detect image-based threats. Among these, Convo-

lutional Neural Networks (CNNs) have demonstrated significant promise due to their robust image recognition capabilities. While the potential of QR code-based phishing is increasingly recognized as a serious threat, there remains a significant gap in research specifically focused on detecting these malicious QR codes within email images.

Studies such as those conducted by Sharevski et al. [2] have shown that a significant number of users tend to scan QR codes without inspecting the embedded URLs for phishing cues, driven by the convenience and immediacy of access. This behavior underscores the importance of developing effective detection mechanisms that do not rely solely on user vigilance. Combining CNNs with traditional security measures could enhance the detection of malicious QR codes, offering a more comprehensive defense against cyber threats.

In this paper, we focus on the application of CNNs in enhancing QR code security within the context of cybersecurity, specifically targeting the detection of QR codes embedded in email images. By training machine learning models to identify and classify QR codes that pose potential security risks, we aim to augment existing email security frameworks. This research bridges the gap between image recognition and phishing detection by integrating CNN-based image processing techniques with traditional cybersecurity methods. Our approach seeks to develop a scalable and efficient solution to counter the emerging threat of QR code-based phishing attacks. By leveraging the strengths of CNNs in image recognition, this study contributes to the broader field of machine learning in cybersecurity, enhancing the ability of email security systems to effectively identify and mitigate malicious QR code activities, thus safeguarding users from growing cyber threats.

II. Related Work

The emergent tactic of utilizing QR codes in phishing campaigns, particularly those targeting user credentials for Microsoft applications and services, is covered extensively by Arntz [3]. The sophistication of these schemes lies in their dual exploitation of technological novelty and user trust. Arntz's analysis details the intricacies of such attacks, revealing the layered deceit that enables the harvesting of user credentials under the guise of convenience. This work highlights the need for innovative countermeasures against QR code-based phishing, laying the groundwork for further exploration of detection methodologies.

Casayuran [4] explores the rising trend of QR code utilization in phishing campaigns, emphasizing the importance of end-user vigilance. Casayuran's research places QR code phishing within the broader context of cyber awareness, underlining how user behavior plays a critical role in the success of such attacks. This work suggests that security solutions must account for human factors, a consideration that is crucial in the development of machine learning models that can adapt to user-centric attack vectors. Our research builds upon Casayuran's insights by integrating these behavioral considerations into a machine learning framework designed specifically for QR code detection.

Dedenok [5] offers a granular view into the prevalence of QR codes in email-based phishing, underscoring the challenge they pose to traditional email security paradigms. By framing QR code phishing as an evolutionary step in cybercriminal tactics, Dedenok's research provides a vital dataset of attack patterns that can inform the training of machine learning algorithms. Our study extends this work by utilizing these datasets to train and validate a convolutional neural network (CNN) model, specifically tuned to detect QR codes in email images.

The study by Tekale [6] provides a comprehensive overview of QR code technologies, including the security features that can be leveraged for and against their misuse. Tekale's exploration of different QR code types and their error correction capabilities offers valuable insights that can be harnessed in machine learning models to better detect malicious QR codes. Integrating this understanding helps refine the CNN's ability to differentiate between benign and malicious QR code patterns.

Niu et al. [1] delve into the security threats posed by QR codes in the context of the Internet of Things (IoT). Their research emphasizes the need for both technical and non-technical security measures, including cryptographic techniques and third-party management to safeguard against QR code-based attacks. This work underscores the critical role of robust QR code detection mechanisms, which align with our study's focus on leveraging CNNs for automated detection and classification of QR codes within email security frameworks.

Sharevski et al. [7] highlight the real-world implications of QR code phishing through studies conducted in naturalistic settings. Their findings indicate a high rate of user interaction with potentially malicious QR codes, driven by persuasive pretexts and a lack of awareness about phishing risks. These insights stress the importance of implementing user-focused security interventions in QR code scanner applications. Our research aims to address these vulnerabilities by developing CNN-based models that can autonomously detect and alert users to the presence of suspicious QR codes embedded in emails.

Atawneh and Aljehani [8] propose a phishing email

detection model using deep learning techniques, contributing to the broader body of research focused on enhancing email security. While their work centers on text-based phishing detection, our study diverges by focusing on the relatively underexplored domain of image-based phishing detection, particularly QR codes, thereby addressing a significant gap in the literature. Additionally, we expand upon the foundational work of Wang et al. [9] by applying CNNs to the specific challenge of detecting QR codes within the context of email-based threats.

Zhang et al. [10] emphasize the importance of rethinking generalization in deep learning, particularly the challenges of training models for diverse tasks. We apply these principles to the task of QR code detection, refining CNN models to enhance their ability to generalize across various QR code formats and contexts within emails.

Wang et al. [9] demonstrate the effectiveness of CNNs in extracting and learning hierarchical features from raw pixel data, a capability critical for distinguishing between benign and malicious QR codes. Our research leverages this approach, adapting CNN architectures to handle the specific challenges posed by QR code detection in the context of cybersecurity, where rapid and accurate image processing is essential.

In a related domain, Bashi et al. [11] explored the application of deep CNNs for RF fingerprinting in WLAN networks to detect router impersonation attacks. Their work illustrates the versatility of CNNs in cybersecurity, particularly in detecting sophisticated network-based threats. Complementing this, our study applies similar deep learning techniques to the detection of malicious QR codes embedded in emails, enhancing the detection capabilities of email security systems.

In contrast to previous works, our study uniquely integrates image recognition techniques with phishing detection frameworks, offering a comprehensive solution to the growing threat of QR code-based phishing. By leveraging the strengths of CNNs in feature extraction and the extensive data from phishing incidents, our research aims to advance email security systems, providing a scalable and effective defense against this novel attack vector.

III. Problem Statement

The rapid evolution of cyber threats presents a significant challenge in the field of cybersecurity, particularly within the domain of email security. As cybercriminals increasingly shift from traditional file-based attacks to more sophisticated phishing techniques, the emergence of QR codes as a deceptive vector has added a new layer of complexity to the threat landscape. This shift highlights the inadequacy of conventional email security mechanisms, which are primarily designed to detect malicious URLs embedded in text. QR codes, often embedded within email images, bypass these defenses, allowing attackers to exploit user trust and curiosity more effectively.

The core issue is the detection and classification of QR codes embedded in emails, a complex task that is compounded by the sheer volume of email traffic and the need for high accuracy in threat detection. The challenge is further compounded by the computational burden of scanning each email for QR codes while maintaining low false positive rates to prevent disruption of legitimate communications. While existing technologies can scan QR codes, their integration into the high-throughput environment of email servers requires a solution that is not only fast and resource-efficient but also scalable to handle the vast quantities of data processed daily.

Further complicating the issue is that distinguishing between benign and malicious QR codes is a nuanced task that demands precision to avoid undermining user trust and workflow efficiency. This research aims to bridge this gap by proposing a solution that leverages the advanced image recognition capabilities of convolutional neural networks (CNNs) to detect and analyze QR codes within email traffic. When implemented in a production environment, the proposed solution would also incorporate an analysis of email metadata to identify patterns indicative of phishing attempts, thereby enhancing the overall effectiveness of email security systems against this emerging threat.

IV. Methodology

To address the challenge of QR code-based phishing attacks, this research focuses on developing a machine learning model that accurately detects QR codes embedded within email images. The core of this approach is a Convolutional Neural Network (CNN) model, chosen for its proven effectiveness in image recognition tasks due to its ability to hierarchically extract features from raw pixel data. This section details the model architecture, training process, data sources, and implementation strategy, with justifications for the specific choices made.

A. CNN Architecture and Implementation

The CNN developed for this study is designed as a binary classification model, specifically to detect and distinguish between images that contain QR codes and those that do not. The following is a detailed description of the architecture, highlighting each layer's configuration, activation functions, and the rationale behind the choices made.

1) Input Layer: The input to the CNN is preprocessed images, resized to 128x128 pixels with three color channels (RGB). The image size is chosen to balance computational efficiency with the ability to retain enough detail for effective feature extraction.
2) Convolutional Layers: The CNN uses multiple convolutional layers to extract features from the input images. The convolution operation as defined by Saxena & Mishra [12] is represented as:

$$(I * K)(x, y) = \sum_m \sum_n I(x+m, y+n) \cdot K(m, n)$$

where I is the input image, K is the kernel (filter), and (x, y) denotes the position of the convolution operation. The first convolutional layer uses 32 filters that are 3×3 in size with ReLU activation. Subsequent layers increase the filter count to 64 and 128 to capture more complex features.
3) Flatten Layer: A flattening layer is used to convert the 2D output from the final convolutional layer into a 1D vector. This transformation is necessary to interface with the fully connected (dense) layers, enabling the model to perform the classification.
4) Fully Connected (Dense) Layers: The fully connected layers serve to aggregate and interpret features extracted by the convolutional layers. Two dense layers are used: the first with 512 neurons and the second with 256 neurons, both employing the ReLU activation function to introduce nonlinearity. These layers process the flattened vector from the convolutional output, identifying complex patterns that indicate the presence of QR codes.
5) Output Layer: This layer uses a single neuron with sigmoid activation that outputs a value between 0 and 1, which represents the probability of the image containing a QR code. The use of this function allows the model to map predictions to a probability, facilitating decision thresholds for classifying the image as containing a QR code or not.
6) Optimizer: The Adam optimizer is used for training and configured with default parameters: learning rate (lr=0.001), beta1 (beta1=0.9), and beta2 (beta2=0.999).
7) Loss Function: Binary cross-entropy is used to measure the difference between the prediction and the actual label (1 for the presence of a QR code, 0 for absence), which guides the optimization process in an effort to minimize error. The binary cross-entropy loss is given by:

$$L = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where N is the number of samples, y_i is the actual label, and $\hat{y}_i$ is the predicted probability.
8) Hyperparameters: The CNN model utilizes a set of carefully chosen hyperparameters to optimize training performance and model generalization. A batch size of 32 is employed to balance memory usage with computational efficiency, facilitating effective training without overburdening system resources. The model is trained across varying epochs, ranging from 10 to 35, to refine its accuracy through multiple iterations. A learning rate of 0.001 is adopted, a standard starting point in CNN training, ensuring stable and gradual weight updates that foster convergence while avoiding oscillations.
9) Data Augmentation: To enhance the robustness of the model and prevent overfitting, data augmentation techniques are applied using the $ImageDataGenerator$ class. The augmentation includes operations such as rotation, width and height shifting, shearing, zooming, and horizontal flipping. These techniques ensure that the model is exposed to a wide variety of scenarios during training, enhancing its ability to generalize to new, unseen data.

The CNN is implemented using Python with TensorFlow and Keras libraries, chosen for their extensive support for deep learning models and ease of use. (See $Appendix\ 1 - CNN\ Architecture$)

B. Data Sources and Preparation

This project utilizes a diverse assortment of sources for training and validation data to ensure a comprehensive learning process. Public repositories such as GitHub, image datasets from Google Images, and dataset compilations from Kaggle provide the initial dataset used primarily in generations one through four. These sources were chosen for their wide availability and the diverse range of QR code images they offer, which is crucial for training a robust model capable of generalizing across various real-world scenarios. When these sources are exhausted, Python scripts utilizing the 'qrcode' library are used to generate additional QR code images. This helps further diversify the training and validation data, ensuring that the model is not overly dependent on any single type of QR code image.

Fig. 1. Example of a QR code similar to those used in the positive dataset.

Fig. 2. Examples of non-QR code images used in the negative dataset: (a) Rubik's cube, (b) barcode, (c) tic-tac-toe board. These images are used to train the model to distinguish QR codes from other patterns and reduce false positives.

To improve the model's robustness and its ability to distinguish QR codes from other patterns, a variety of visually similar but non-QR code images were included in the negative dataset. These images, such as barcodes, Rubik's cubes, and tic-tac-toe boards, are strategically chosen to address the common challenge of false positives in binary classification tasks. This augmentation step enhances the model's ability to accurately differentiate between QR codes and other similar visual elements.

All images undergo a preprocessing step that involves resizing them to a consistent dimension of 150x150 pixels to ensure uniform input size for the model. The inclusion of diverse images in both the positive and negative datasets plays a crucial role in training the CNN. By exposing the model to a broad spectrum of patterns, the training process helps develop a feature recognition capability that can be more easily generalized. This strategy not only improves the accuracy of detecting legitimate QR codes but also minimizes the chances of misclassifying non-QR code patterns as malicious.

C. Data Augmentation and Training Process

The data augmentation process uses Python libraries such as $ImageDataGenerator$ to generate variations of existing images through rotation, scaling, and flipping. These transformations expose the model to a wide range of image variations during training, which is essential for improving the model's robustness and reducing overfitting.

The CNN is trained using the augmented dataset in an iterative manner. The training process involves several cycles where the model is exposed to progressively more diverse image sets. The Adam optimizer is used for efficient gradient descent, and was selected due to its efficiency in handling large datasets and its proven effectiveness in deep learning tasks. (See $Appendix$ 2 $-$ $Training\ Process$)

D. Evaluation and Validation

To assess the model's performance, a separate validation dataset is used that was not included in the training phase. This approach simulates a real-world scenario where the model encounters new, unseen data.

E. Efforts Toward Continuous Improvement

Throughout the project, detailed reports are generated after each training iteration, documenting key performance metrics and providing insights into areas of improvement. These reports are critical for ensuring a continuous improvement loop, where insights from each iteration inform subsequent training cycles. The methodology outlined here aims to create a robust, scalable solution for detecting QR code-based phishing attacks. By leveraging the power of CNNs and comprehensive data augmentation, this research endeavors to contribute a significant advancement in email security.

V. Experimentation

This experiment was conducted over seven iterative training generations of a CNN model, each designed to optimize the detection of QR codes in images. The goal was to evaluate how variations in dataset size, model complexity, and training duration impacted the model's ability to generalize effectively and maintain high accuracy in real-world scenarios.

A. Experimental Setup and Parameters

The experiment started with a relatively small dataset and fewer epochs, progressively scaling up in subsequent generations to test the model's ability to handle larger datasets and more complex training scenarios. Key parameters such as the number of training images, validation images, epochs, and steps per epoch were adjusted across generations as detailed in $Appendix$ 3 $-$ $Model\ Training$.

B. Observations and Analysis

The first six generations demonstrated consistently high training and validation accuracy, indicating that the model was effectively learning to distinguish QR codes from other image patterns. However, in the seventh generation, despite using the largest dataset (200,000 training images) and the most extensive training (35

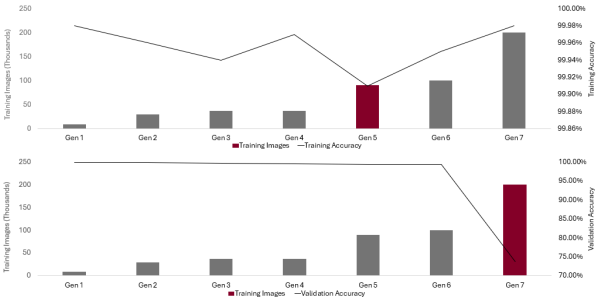

Fig. 3. Training and Validation Accuracy over Epochs. These graphs show the trend of accuracy improvement and the point at which overfitting may have occurred, as evidenced by the divergence between training and validation accuracy.

epochs), the model exhibited a significant drop in validation accuracy from 99.44% to 73.70%.

This drop suggests that the model may have overfitted to the training data. The increasing number of epochs and the larger dataset in Generation 7 likely exacerbated this issue, as the model continued to optimize for the training set while losing flexibility in applying its learned features to the validation set. To address this, future experimentation should focus on implementing techniques to mitigate overfitting. These could include:

1) Regularization Techniques: Adding L2 regularization or dropout layers to the model to penalize overly complex models.
2) Data Augmentation: Further increasing the diversity of the training data through advanced data augmentation techniques to prevent the model from memorizing specific patterns.
3) Early Stopping: Implementing early stopping based on validation loss or accuracy to prevent the model from overtraining.
4) Choice of Hyperparameters: Learning rate, batch size, and architecture of the CNN could be revisited to fine-tune the balance between model complexity and generalization capability.

VI. Results and Future Work

Figure 3 presents a graphical representation of the training and validation accuracy across different epochs, highlighting the trends in model performance and the potential overfitting observed in the later stages of experimentation.

The experimentation with the CNN model across seven generations demonstrated both progress and challenges. The model's training accuracy consistently improved, achieving near-perfect results of 99.98% by the seventh generation. However, the significant drop in validation accuracy in the final generation suggests that the model's capacity was stretched to a point where its ability to generalize was compromised, indicating potential overfitting.

These results highlight a key insight: while increasing the complexity of the model and expanding the dataset can improve training accuracy, they do not necessarily lead to better generalization. This calls for a reconsideration of our approach in future experimentation. Given the overfitting observed in the final generation, exploring alternative model architectures that inherently resist overfitting, such as deeper networks with dropout layers, will be crucial. Additionally, incorporating real-world testing will not only validate the model's performance but also help identify any unforeseen challenges when deployed in live environments.

A. Future Work Directions

1) Exploration of Alternative Model Architectures: Given the limitations observed with the current CNN architecture in the seventh generation, future work could explore alternative architectures that may offer better generalization capabilities. This could involve experimenting with different types of neural networks or hybrid models that combine CNNs with other techniques.
2) Incorporation of Real-World Testing: Deploying the model in controlled environments that simulate actual email traffic can provide deeper insights into its practical performance, helping to identify potential gaps that were not apparent during initial experimentation.
3) Focus on Model Interpretability: As the model becomes more complex, understanding how it makes decisions becomes increasingly important. Future efforts could include developing methods to interpret the CNN's decisions, ensuring that the model's predictions are not only accurate but also understandable and explainable.
4) Longitudinal Performance Monitoring: Continuous monitoring of the model's performance over time, especially after deployment, will be essential. This will help in identifying any degradation in accuracy or emerging patterns that could indicate new types of phishing tactics, allowing for timely updates to the model.
5) Collaboration with Industry Partners: To ensure that the model is aligned with current industry needs, future research could seek to collaborate with cybersecurity firms. This collaboration may provide access to more diverse datasets and insights into the latest threat landscapes, enabling the model to be more robust and effective.

The results of this study underscore the complexity of developing a model that is both highly accurate and

generalizable. While the initial results were promising, the challenges identified in the later stages of experimentation highlight areas for further investigation. By pursuing the proposed directions for future work, the research aims to enhance the model's robustness and applicability, ultimately contributing to more secure and resilient email systems against QR code-based phishing threats.

This study contributes to the field by introducing a novel approach that integrates CNNs with traditional email security systems, specifically targeting the detection of QR code-based phishing threats. The insights gained from this research provide a foundation for further developments in adaptive, image-based threat detection technologies.

VII. Discussion

While the results of this research are promising, particularly in demonstrating the capability of machine learning models to identify QR codes within email images, this is only one piece of a broader security framework. The detection of QR codes is a crucial first step, but it must be integrated with other security measures to effectively mitigate phishing threats. Specifically, URL analysis plays a pivotal role in identifying sites that are commonly exploited by threat actors. Free file hosting sites, for instance, are often misused for malicious purposes due to their ease of access and the challenges associated with tracing and removing harmful content.

Phishing campaigns frequently deploy fake login pages that mimic those of trusted brands like Microsoft and Google to deceive users into divulging their credentials. To counter this, the QR code detection model could be augmented with heuristic analysis techniques that scrutinize the associated URLs and web content. Key indicators such as the domain of the URL, the presence of SSL certificates, and the visual layout of the landing page could provide valuable insights into the legitimacy of a site. Additionally, linguistic analysis of the email content could uncover social engineering tactics, such as urgent calls to action or offers that seem suspiciously advantageous.

The adaptability of this model is crucial. As phishing tactics continue to evolve, the model must also be continuously updated and refined. This involves not only regular retraining with new data but also integrating feedback loops that allow the model to learn from the latest phishing trends. Collaboration with threat intelligence networks could further enhance the model's ability to counter new and emerging threats, ensuring it remains a robust component of a comprehensive cybersecurity strategy.

VIII. Conclusion

The rapid evolution of phishing tactics, particularly the rise of QR code-based threats, underscores the necessity for a multifaceted and adaptive defense strategy in cybersecurity. This research demonstrates the potential of integrating machine learning-based image recognition techniques, such as convolutional neural networks, with comprehensive detection frameworks to effectively counter these emerging threats. However, it also highlights that such models must be part of a broader security ensemble that includes traditional detection methods, such as sender reputation evaluation, forensic analysis of message headers, and content inspection.

The success of machine learning models in this domain is heavily dependent on the quality and relevance of the training data. High-fidelity datasets that accurately represent real-world scenarios are crucial for developing models that not only detect QR codes but can also discern the context in which they are used in phishing attempts. By leveraging data that reflects the diverse tactics employed by cybercriminals, these models can be fine-tuned to predict and counteract a wide range of threats.

In addition to deploying advanced detection systems, organizations must adopt and enforce robust email security practices. Implementing protocols like DMARC (Domain-based Message Authentication, Reporting & Conformance) can help authenticate email sources and create feedback loops for identifying suspicious activity. Incorporating the human element into the security strategy through regular, targeted security awareness training is essential. Educating users about the complexities of modern phishing attacks, including those involving QR codes, equips them to act as an additional line of defense.

Layering security measures across various domains—such as deploying browser isolation solutions, utilizing secure enterprise browsers, and enforcing multi-factor authentication—can further strengthen an organization's defenses. These measures, when combined with the findings from this research, create a resilient security posture capable of withstanding the dynamic and increasingly sophisticated threats in today's digital landscape. This research serves as a foundational step towards developing effective defenses against image-based phishing attacks. The insights gained here provide a blueprint for organizations to enhance their cybersecurity frameworks, ensuring they are better equipped to combat the ever-evolving nature of email-based threats.

REFERENCES

[1] Niu, X., Zhao, J., & Tian, B. (2024). View of the security threat and precautionary measures of QR code of internet of things technology. Advances in Engineering Technology Research, 11, 2790–1688. https://doi.org/10.56028/aetr.11.1.775.2024

[2] Sharevski, F., Mossano, M., Veit, M. F., Schiefer, G., & Volkamer, M. (2024). Exploring Phishing Threats through QR Codes in Naturalistic Settings - NDSS Symposium [Symposium on Usable Security and Privacy (USEC) 2024]. https://doi.org/10.14722/usec.2024.23050

[3] Arntz, P. (2023, August 20). QR codes used to phish for Microsoft credentials. Malwarebytes. https://www.malwarebytes.com/blog/news/2023/08/qr-codes-deployed-in-targeted-phishing-campaigns

[4] Casayuran, M. (2023, October 5). Think Before You Scan: The Rise of QR Codes in Phishing. Spiderlabs Blog. https://www.trustwave.com/en-us/resources/blogs/spiderlabs-blog/think-before

[5] Dedenok, R. (2023, September 27). QR codes in email phishing. Securelist by Kaspersky. https://securelist.com/qr-codes-in-phishing/110676/

[6] Tekale, H. A. (2024). A Comprehensive Study on QR Codes (3rd ed., Vol. 4) [International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)]. https://doi.org/10.48175/IJARSCT-18935

[7] Sharevski, F., Mossano, M., Veit, M. F., Schiefer, G., & Volkamer, M. (2024). Exploring Phishing Threats through QR Codes in Naturalistic Settings - NDSS Symposium [Symposium on Usable Security and Privacy (USEC) 2024]. https://doi.org/10.14722/usec.2024.23050

[8] Atawneh, S., & Aljehani, H. (2023). Phishing Email Detection Model Using Deep Learning. Electronics 2023, 12(20), 4261. https://doi.org/10.3390/electronics12204261

[9] Wang, J., Zeng, X., Duan, S., Zhou, Q., & Peng, H. (2022). Image target recognition based on improved convolutional neural network. Mathematical Problems in Engineering, 2022, 1–11. https://doi.org/10.1155/2022/2213295

[10] Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2021). Understanding deep learning (still) requires rethinking generalization. Communications of the ACM, 64(3), 107–115. https://doi.org/10.1145/3446776

[11] Bashi, O. I. D., Jameel, S. M., Kubaisi, Y. M. A., Hameed, H. K., & Sabry, A. H. (2023). Threat detection model for WLAN of simulated data using deep convolutional neural network. Applied Sciences, 13(20), 11592. https://doi.org/10.3390/app132011592

[12] Saxena, A. K., & Mishra, P. P. (2022, November 19). Optimizing Electric Vehicle Energy Management Systems with a Hybrid LSTM-CNN Architecture. https://research.tensorgate.org/index.php/tjstidc/article/view/108

IX. Appendix 1 - CNN Architecture

```
# Construct the CNN model for binary classification
model = tf.keras.models.Sequential([
    # Convolutional layer with 32 filters, 3x3 kernel size, and ReLU activation function
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(150, 150, 3)),
    # MaxPooling layer to reduce spatial dimensions
    tf.keras.layers.MaxPooling2D(2, 2),
    # Repeat the above two layers, increasing filter sizes
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    # Flatten layer to convert 2D matrix to vector for Dense layers
    tf.keras.layers.Flatten(),
    # Dense layer with 512 units and ReLU activation
    tf.keras.layers.Dense(512, activation='relu'),
    # Output layer with 1 unit and sigmoid activation for binary classification
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

X. Appendix 2 - Training Process

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data augmentation setup
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_generator = train_datagen.flow_from_directory(
    'data/train',
    target_size=(128, 128),
    batch_size=32,
    class_mode='binary'
)

# Model training
history = model.fit(
    train_generator,
    steps_per_epoch=100,
    epochs=20,
    validation_data=validation_generator,
    validation_steps=50
)
```

XI. Appendix 3 - Model Training

	Gen 1	*Gen* 2	*Gen* 3	*Gen* 4	*Gen* 5	*Gen* 6	*Gen* 7
Training Images	9000	29453	36954	37493	90113	99916	200000
Validation Images	100	991	1103	1206	2213	3214	10000
Epochs	10	15	15	20	20	25	35
Training Steps	100	100	100	150	150	200	250
Validation Steps	50	50	50	50	50	100	200
Validation Accuracy	99.91%	99.81%	99.75%	99.50%	99.37%	99.44%	73.70%
Training Accuracy	99.98%	99.96%	99.94%	99.97%	99.91%	99.95%	99.98%

View publication stats